

Apuntes Inteligencia Artificial

Anthony Guevara Guillén
Escuela de Ingeniería en Computación
Tecnológico de Costa Rica
Carné: 2022019397

Profesor: Steven Pacheco Portuquez
Curso: IC6200 Inteligencia Artificial
Fecha: 28 de abril del 2026

Resumen—El presente documento resume los apuntes tomados durante la clase del 21 de abril de 2026 para el curso IC6200 Inteligencia Artificial. Se abordan temas fundamentales de redes neuronales, incluyendo propagación hacia adelante, funciones de activación, cálculo de pérdida mediante entropía cruzada y los fundamentos de optimización por retropropagación.

I. RESPUESTAS DEL QUIZ

I-A. Describa qué es una red totalmente conectada incluyendo sus 3 capas (Fully-connected) [30 pts]

Consiste en una red compleja donde se pasa por funciones no lineales para llegar a la salida. Se compone por las siguientes capas:

- **Capa de entrada:** Contiene las características (*features*) iniciales.
- **Capa intermedia (oculta):** Compuesta por una o varias funciones de activación.
- **Capa de salida:** Contiene la función de activación final encargada de la clasificación.

I-B. ¿Qué es un one-hot vector y cómo me ayuda a resolver problemas de regresión multinomial? [30 pts]

Se trata de una codificación donde se enciende un bit correspondiente a la entrada registrada (marcando la clase con 1 y el resto con 0). Se comparan los vectores con el fin de tener una salida de múltiples regresiones, permitiendo así manejar probabilidades independientes para múltiples clases.

I-C. Mencione tres técnicas de aumentación de datos para señales de audio propuestas en SpecAugment [30 pts]

Las tres técnicas son:

- *Time Warping*
- *Frequency Masking*
- *Time Masking*

I-D. ¿Por qué preferimos resolver problemas de clasificación binaria con una red neuronal de mínimo una capa oculta antes que una regresión logística? [10 pts]

Se prefiere por la necesidad de manejar una relación no lineal respecto a nuestra entrada. El uso de una red neuronal con al menos una capa oculta permite la resolución de problemas no lineales, lo que significa que es capaz de abordar un set de problemas mucho más complejo, contrario a las limitaciones geométricas que presenta una simple regresión logística.

II. REPASO CLASE JUEVES

II-A. Perceptrón

El perceptrón es básicamente una regresión logística, pero su problema fue que producía comparaciones lineales y solo funcionaba con problemas lineales. Esto provocó el invierno de la investigación con inteligencia artificial en los años 70, pues no se podían solucionar problemas lógicos complejos como el XOR. Fue sino hasta la llegada de redes neuronales más profundas y el algoritmo de *Backpropagation* que hubo un avance significativo.

II-B. Redes Neuronales e Inspiración Biológica

Las redes neuronales artificiales tienen una fuerte inspiración biológica. De forma similar a nuestro sistema nervioso, la neurona artificial recibe una señal de entrada y, mediante una función de activación, decide bloquearla, transformarla o dejarla pasar hacia la siguiente capa según la intensidad computada.

II-C. Distribución de Jerarquía y Maldición de Dimensionalidad

Para lograr el aprendizaje, se mantiene una búsqueda de comportamiento jerárquico. En las primeras capas se asimilan características simples (líneas, colores) y, a medida que se avanza en profundidad, se extraen elementos semánticos más específicos (como rostros).

Si se escoge una cantidad de neuronas y capas demasiado grandes, se aumenta la dimensionalidad del problema, dificultando hallar una función que optimice el costo (Maldición de la dimensionalidad). Para mitigar el costo computacional de dimensiones altas en otros dominios (como texto), por medio de *embeddings* se puede mapear la información en vectores (ej. 512 caracteres) que guardan el valor semántico, permitiendo usar algoritmos de similitud (como la distancia de coseno).

II-D. Redes Convolucionales (CNN)

A diferencia de una red *fully-connected*, la arquitectura convolucional consiste en un pequeño kernel que se desliza a través de la imagen para extraer características (bordes, sombras, colores). Conforme avanza la arquitectura, obtiene mayores detalles.

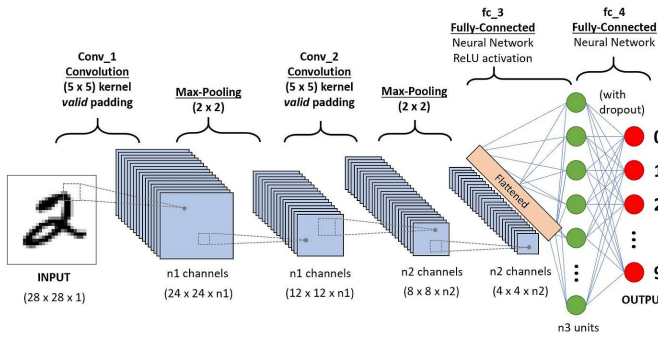


Figura 1. Arquitectura típica de una Red Neuronal Convolutional (CNN) mostrando la extracción de características mediante kernels [1].

III. PROPAGACIÓN HACIA ADELANTE

En las redes neuronales multicapa, la computación se realiza de forma anidada a través de las diferentes capas ocultas, funcionando de manera muy similar a una composición de funciones matemáticas. Para llegar a la capa final (denotada como capa N o capa de salida), se debe computar la función de activación (como la función sigmoide) y todas las capas anteriores (por ejemplo, $H_{N-1}, \dots, H_2, H_1, H_0$).

Existe una relación lineal entre los pesos de una capa específica y la salida de la capa anterior. Cada una de estas salidas intermedias se transforma y toma forma mediante las funciones de activación aplicadas en esa capa.

IV. CLASIFICACIÓN CON PROBABILIDADES

Cada neurona en la capa de salida produce valores independientes que oscilan entre 0 y 1. En un problema de clasificación, cada una de estas neuronas representa la probabilidad de que la entrada pertenezca a una clase específica (por ejemplo, Clase 1, Clase 2, etc.).

Es fundamental que los valores obtenidos en la capa de salida formen en conjunto una distribución de probabilidades válida, lo que significa que la suma de todas las probabilidades de salida debe ser exactamente 1. Durante el proceso de inferencia, la clase que obtenga la mayor probabilidad se selecciona como la predicción final del modelo.

V. ONE-HOT ENCODING Y COMPARACIÓN

Para representar las categorías a predecir, se utiliza la codificación *One-Hot* (One-Hot vector). La clase predicha por el modelo se marca con un 1 y el resto de las clases con 0, creando así un vector posicional.

Este vector resultante se compara directamente con el vector *One-Hot* de la etiqueta verdadera (la cual debe ser transformada previamente a este formato para el entrenamiento). Si el valor de 1 se encuentra en la misma posición en ambos vectores, se determina que la clasificación es correcta; si están en posiciones diferentes, la predicción es rechazada o considerada errónea.

VI. FUNCIONES DE ACTIVACIÓN Y PÉRDIDA

La función de activación agrega comportamiento a la red y permite acotar las salidas infinitas de cada capa para controlar el resultado final.

Es importante destacar que la capa de salida no necesariamente utiliza una función Sigmoide. Dado que se requiere estrictamente una distribución de probabilidades (donde las salidas sumen 1), se requiere una función adecuada (como Softmax, por ejemplo).

Además, se debe definir una función de pérdida (*Loss Function*) apropiada para evaluar estas distribuciones. La *Entropía Cruzada* (*Cross-Entropy*) es la función de pérdida estándar y con mayor sentido en este contexto, ya que evalúa qué tan bien se ajustan las probabilidades predichas a las reales. Funciones como el Error Cuadrático Medio (*Mean Squared Error* o MSE) no son apropiadas para problemas de clasificación probabilística, ya que no siguen una distribución de este tipo.

A continuación, se detallan las principales funciones de activación abordadas:

VI-A. Función Lineal

Si bien funciona en problemas simples, el problema principal es que su gradiente siempre es constante, lo que impide descender de manera óptima sobre la función de costo. Además, produce salidas hacia el infinito y no añade el nivel de complejidad no lineal requerido para redes multicapa.

$$y = mx + b \quad (1)$$

$$\frac{d}{dx}(mx + b) = m \quad (2)$$

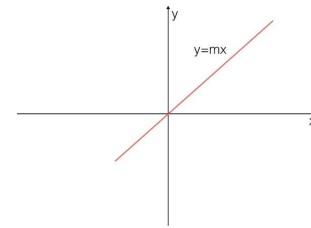


Figura 2. Gráfica de la función lineal. Presenta un gradiente constante.

VI-B. Función Sigmoide

Acota los valores entre 0 y 1 y siempre es positiva. El problema se encuentra en los extremos de la función, donde su valor es casi constante y la derivada se acerca a cero. Esto genera gradientes muy pequeños (*vanishing gradient*) que impiden avanzar significativamente en la función de costo.

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (3)$$

$$\frac{d\sigma(x)}{dx} = \sigma(x)(1 - \sigma(x)) \quad (4)$$

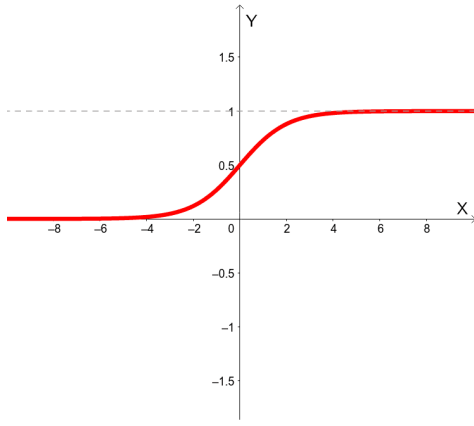


Figura 3. Comportamiento de la función Sigmoide.

VI-C. Función Tangente Hiperbólica (Tanh)

Similar a la sigmoide, pero acota los valores entre -1 y 1. Presenta exactamente el mismo problema en los extremos: el gradiente es muy cercano a cero, limitando la optimización de los pesos.

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad (5)$$

$$\frac{d}{dx} \tanh(x) = 1 - \tanh^2(x) \quad (6)$$

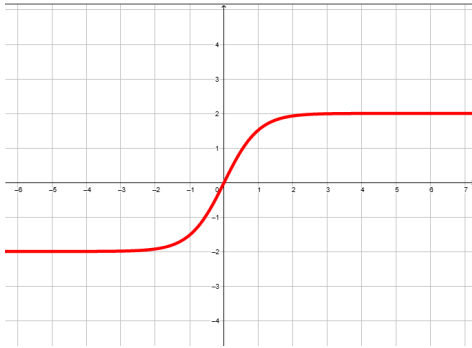


Figura 4. Comportamiento de la función Tanh.

VI-D. Función ReLU (Rectified Linear Unit)

Se define como el máximo entre x y 0; la parte negativa es constante (0) y la parte positiva es el mismo valor de x . Es sumamente eficiente para *Deep Learning*. Sin embargo, presenta el problema de las *neuronas muertas*: si los valores caen a la izquierda del cero, el gradiente es cero absoluto y esos pesos nunca se actualizan.

$$\text{ReLU}(x) = \max(0, x) \quad (7)$$

$$\frac{d}{dx} \text{ReLU}(x) = \begin{cases} 0, & x < 0 \\ 1, & x > 0 \end{cases} \quad (8)$$

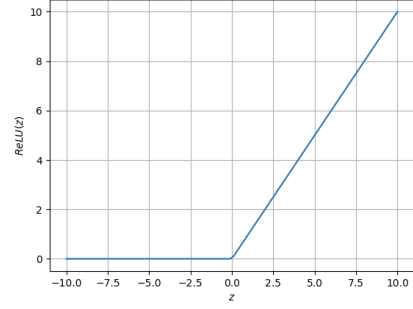


Figura 5. Función ReLU. Eficiente computacionalmente, pero propensa a generar neuronas muertas en su segmento negativo [4].

VI-E. Leaky ReLU

Es una alternativa para solucionar el problema de las neuronas muertas en ReLU. Multiplica los valores negativos por una constante muy pequeña (ej. 0,001), permitiendo que exista un mínimo gradiente y la red siga aprendiendo. Su desventaja es que esta constante es un valor arbitrario fijo que podría no ser el óptimo para todos los problemas.

$$\text{LeakyReLU}(x) = \begin{cases} x, & x \geq 0 \\ 0,001x, & x < 0 \end{cases} \quad (9)$$

$$\frac{d}{dx} \text{LeakyReLU}(x) = \begin{cases} 1, & x \geq 0 \\ 0,001, & x < 0 \end{cases} \quad (10)$$

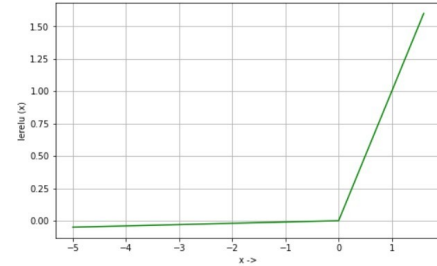


Figura 6. Función Leaky ReLU [3].

VI-F. Parametric ReLU

Soluciona la limitación de Leaky ReLU permitiendo que la red *aprenda* el parámetro de la constante (W) durante el entrenamiento. Es más dinámica y se ajusta mejor a los datos, aunque aumenta la complejidad computacional al tener más pesos que optimizar.

$$\text{PReLU}(x) = \begin{cases} x, & x \geq 0 \\ ax, & x < 0 \end{cases} \quad (11)$$

$$\frac{d}{dx} \text{PReLU}(x) = \begin{cases} 1, & x \geq 0 \\ a, & x < 0 \end{cases} \quad (12)$$

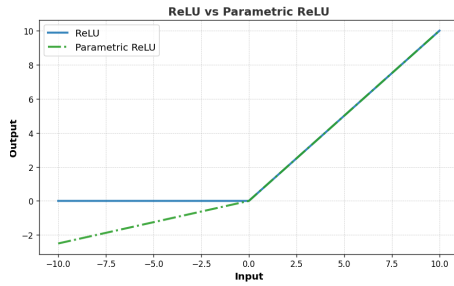


Figura 7. Función Parametric ReLU vs. ReLU [3].

VI-G. Soft Max

Se utiliza para convertir los valores crudos de salida (*logits*) en una distribución de probabilidades estandarizada. Su cálculo exponencial asegura que la sumatoria de las probabilidades de todas las clases sea exactamente 1. Se emplea en conjunto con la función de pérdida *Cross-Entropy*.

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \quad (13)$$

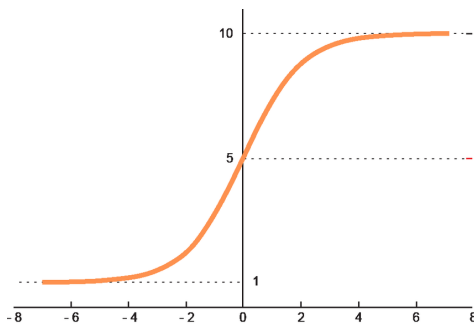


Figura 8. Función SoftMax [5].

VII. ¿CUÁL FUNCIÓN DE ACTIVACIÓN UTILIZAR?

Depende del requerimiento del modelo. La recomendación general es empezar implementando **ReLU**. Si ReLU presenta problemas (neuronas muertas), se puede intentar con Leaky ReLU o Parametric ReLU. Las funciones Sigmoides y Tanh no se recomiendan para capas ocultas intermedias debido a los gradientes minúsculos, aunque Tanh solía usarse mucho en modelos de lenguaje recurrentes (Recurrent Neural networks (RNN) o Long Short-Term Memory (LSTM)) más antiguos.

VIII. OPTIMIZACIÓN Y BACKPROPAGATION

La optimización de los pesos de la red neuronal se realiza utilizando el algoritmo de *Descenso del Gradiente* (*Gradient Descent*).

Para optimizar correctamente la red a través de la propagación hacia atrás, es necesario cuantificar cuánto afecta cada neurona a la función de pérdida final. Para ello, se calcula la derivada de la función de costo con respecto a los pesos en cada neurona, obteniendo el nivel de contribución al error general.

El tipo de capas internas utilizadas (ya sean convolucionales, con distintas activaciones, etc.) actúa primordialmente como un mecanismo de extracción de características pegado a una superficie y no altera el mecanismo de la capa de salida.

IX. USO DE UNA RED NEURONAL COMO REGRESIÓN

Si se tiene una red neuronal gigante y se necesita que el modelo prediga un valor numérico específico (regresión), lo único que se debe hacer es aplicar una **activación lineal** (o directamente no usar ninguna función de activación) en la capa de salida. De esta forma, el modelo retorna directamente los *logits* calculados por $wx + b$.

X. COMENTARIOS SOBRE EL PROYECTO

El proyecto tiene total libertad de lenguaje, *framework* o interfaz visual para la programación de la aplicación. La entrega está programada para el jueves 7 de mayo (con posibilidad de moverse un par de días), y las revisiones técnicas iniciarán la semana del lunes 11 de mayo. Se espera justificar la experimentación realizada con las arquitecturas seleccionadas.

REFERENCIAS

- [1] E. Canorea, "Convolutional Neural Network: Guía para entenderlas," *Plain Concepts*, 13 de jun. de 2025. [En línea]. Disponible: <https://www.plainconcepts.com/es/convolutional-neural-network>
- [2] Codificando Bits, "La Función de Activación," *Codificando Bits*, 21 de feb. de 2024. [En línea]. Disponible: <https://codificandobits.com/blog/funcion-de-activacion/>
- [3] GeeksforGeeks, "Leaky Relu activation function in deep learning," *GeeksforGeeks*, 12 de jul. de 2025. [En línea]. Disponible: <https://www.geeksforgeeks.org/machine-learning/leaky-relu-activation-function/>
- [4] GeeksforGeeks, "ReLU activation function in Deep learning," *GeeksforGeeks*, 15 de abr. de 2026. [En línea]. Disponible: <https://www.geeksforgeeks.org/deep-learning/relu-activation-function/>
- [5] J. Qadir, "Graphic representation of the softmax activation function," *ResearchGate*, 2021. [En línea]. Disponible: <https://www.researchgate.net/figure/Graphic-representation-of-the-softmax-activation-function>